COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07/28/2026	**DATE SUBMITTED:**	07/31/2026
NAME:	Lance Cristopher S. Salcedo	**INSTRUCTOR:**	Engr. Jimlord Quejardo

HANDS ON ACTIVITY #3.1 **Linked Lists**

1. PROCEDURE OUTPUT

Screenshot

main.cpp

```cpp
main.cpp > ...
#include <iostream>
#include <utility>
#include "singly_LL.h"

int main(){
//step 1
    Node<char> *head = NULL;
    Node<char> *second = NULL;
    Node<char> *third = NULL;
    Node<char> *fourth = NULL;
    Node<char> *fifth = NULL;
    Node<char> *last = NULL;
//step 2
    head = new Node<char>;
    second = new Node<char>;
    third = new Node<char>;
    fourth = new Node<char>;
    fifth = new Node<char>;
    last = new Node<char>;
//step 3
    head->data = 'C';
    head->next = second;

    second->data = 'P';
    second->next = third;

    third->data = 'E';
    third->next = fourth;

    fourth->data = '1';
    fourth->next = fifth;

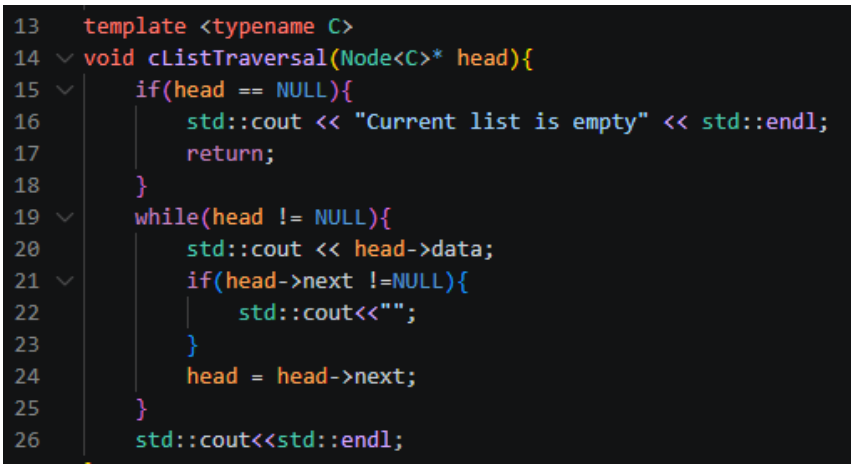
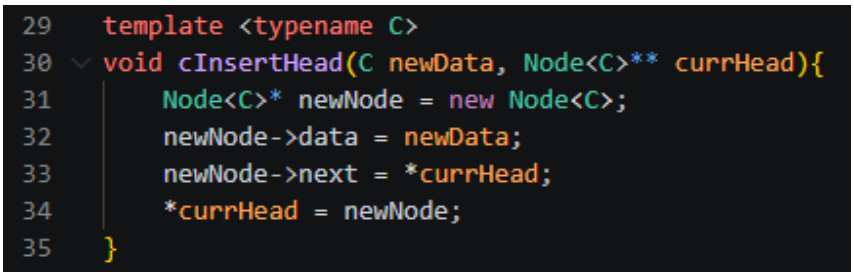
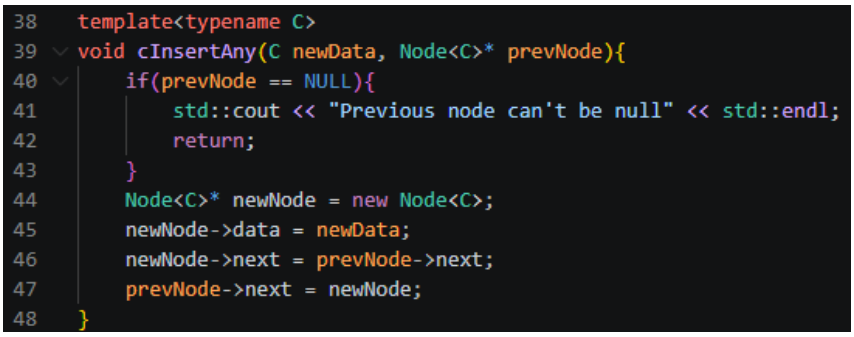
    fifth->data = '0';
    fifth->next = last;
//step 4
    last->data = '1';
    last->next = nullptr;
```

Discussion

In the code that was given, in the main function, it has node pointers that are currently equivalent to null in step 1. In step 2, the code was creating the nodes based on the six node pointers, each node storing characters in the given code (char), In the step 3, each node has two parts which are data and next, data which stores the character (char) and next being the address from head to last. In step 4, in the last node the code declares it as a nullptr meaning there is no next node.

Table 3-1. Output of Initial/Simple Implementation

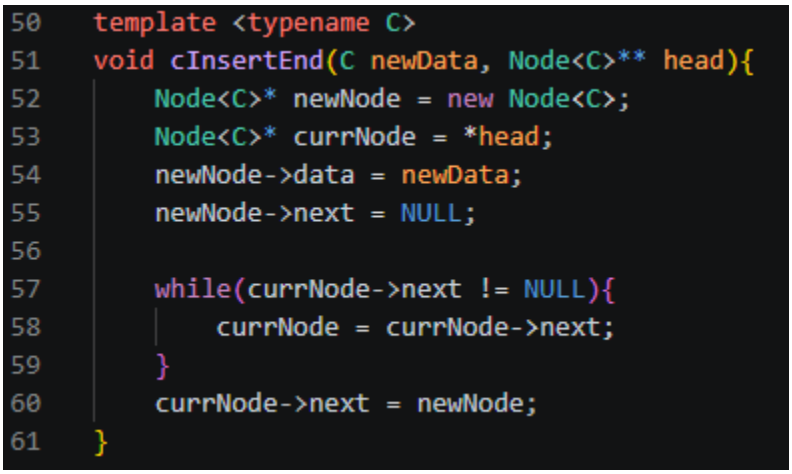
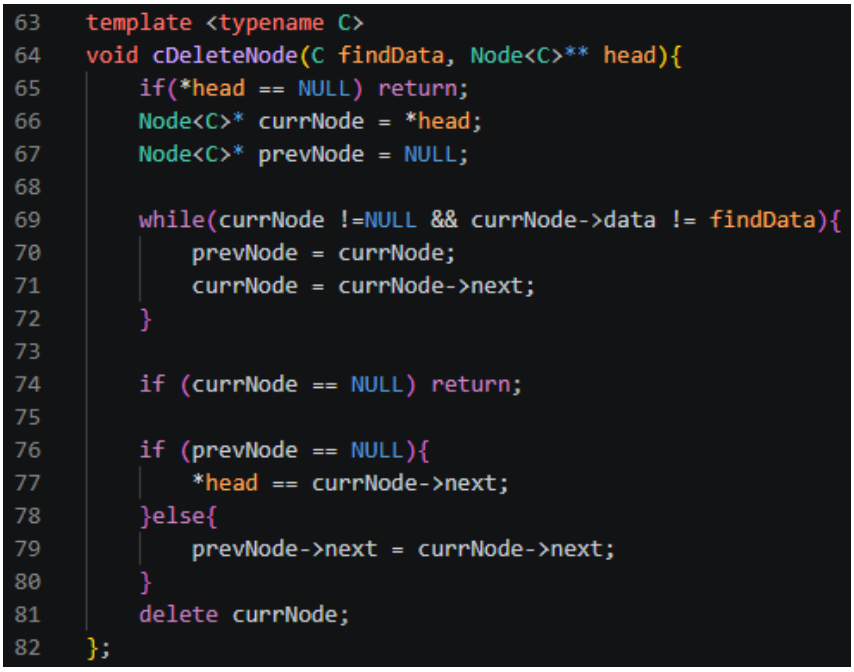
Operation	Screenshot

Traversal	
Insertion at head	
Insertion at any part of the list	

```
13  template <typename C>
14  void cListTraversal(Node<C>* head){
15      if(head == NULL){
16          std::cout << "Current list is empty" << std::endl;
17          return;
18      }
19      while(head != NULL){
20          std::cout << head->data;
21          if(head->next !=NULL){
22              std::cout<<"";
23          }
24          head = head->next;
25      }
26      std::cout<<std::endl;
```

```
29  template <typename C>
30  void cInsertHead(C newData, Node<C>** currHead){
31      Node<C>* newNode = new Node<C>;
32      newNode->data = newData;
33      newNode->next = *currHead;
34      *currHead = newNode;
35  }
```

```
38  template<typename C>
39  void cInsertAny(C newData, Node<C>* prevNode){
40      if(prevNode == NULL){
41          std::cout << "Previous node can't be null" << std::endl;
42          return;
43      }
44      Node<C>* newNode = new Node<C>;
45      newNode->data = newData;
46      newNode->next = prevNode->next;
47      prevNode->next = newNode;
48  }
```

Insertion at the end	
Deletion of a node	

```
50  template <typename C>
51  void cInsertEnd(C newData, Node<C>** head){
52      Node<C>* newNode = new Node<C>;
53      Node<C>* currNode = *head;
54      newNode->data = newData;
55      newNode->next = NULL;
56
57      while(currNode->next != NULL){
58          currNode = currNode->next;
59      }
60      currNode->next = newNode;
61  }
```

```
63  template <typename C>
64  void cDeleteNode(C findData, Node<C>** head){
65      if(*head == NULL) return;
66      Node<C>* currNode = *head;
67      Node<C>* prevNode = NULL;
68
69      while(currNode !=NULL && currNode->data != findData){
70          prevNode = currNode;
71          currNode = currNode->next;
72      }
73
74      if (currNode == NULL) return;
75
76      if (prevNode == NULL){
77          *head == currNode->next;
78      }else{
79          prevNode->next = currNode->next;
80      }
81      delete currNode;
82  };
```

Table 3-2. Code for the List Operations

a.	**Source Code Console**	Code:
		`40 std::cout << "Original List: " << std::endl;`
		`41 cListTraversal(head);`
		Output:
		`Original List:`
		`CPE101`
		a. Traverse the list by passing the head of the created list into the function
		Analysis:
		In the singly file, I used template <typename C> and used void to only run it once, it checks if the list is empty, if it doesn't it will continue to loop as long as the head goes to a node. It also prints the data of the current node, then stores it in the next node. In the main function, I used the void function to show the traversal showing it as "CPE101".

b.	**Source Code Console**	Code:
		`43 std::cout << "\nInserting 'G' node in the head: " << std::endl;`
		`44 Node<char>* newNode = new Node<char>;`
		`45 cInsertHead('G', &head);`
		`46 cListTraversal(head);`
		Output:
		`Inserting 'G' node in the head:`
		`GCPE101`
		b. Insert the element 'G' at the start of the list to replace the current node. Output should now show "GCPE101"
		Analysis:
		In the singly file, I used template <typename C> and used void to only run it once, it first creates a new node, then stores it, linking the new node to the old head so it would make it the new head. In the main function, I insert G to the new node making it print alongside 'CPE101'.

c.	**Source Code Console**	Code:

```
48      std::cout << "\nInserting another 'E' node at the third: "
49      << std::endl;
50      Node<char> *current = head;
51      while(current!= 0 && current->data !='P'){
52          current = current->next;
53      }
54      if(current != 0){
55          cInsertAny('E', current);
56      }
57      cListTraversal(head);
```

Output:

```
Inserting another 'E' node at the third:
GCPEE101
```

c. Insert an element "E" with the previous node element being "P". Output should now show "GCPEE101".

Analysis:

Asides from making the template and checking the error, I created a new node, put that data into the node, and connected it. In the main function, I created a temporary variable to hold as a placeholder, then it searches for the node that contains 'P' then insert 'E' into it, so when it prints it becomes 'GCPEE101'.

d. **Source Code Console**

Code:

```
59      std::cout <<"\nDeleting the node 'C': " << std::endl;
60      cDeleteNode('C', &head);
61      cListTraversal(head);
```

Output:

```
Deleting the node 'C':
GPEE101
```

d. Delete the node containing the element C.

Analysis:

Asides from checking the error and making the template, it first searches for the node that needs to be deleted, if the node that needs to be deleted wasn't there it returns, it also has a lot of if/else statements depending if the node that needs to be deleted are either the first node, middle or last. In the main function, the selected node is letter 'C', it searches, updates and prints it as 'GPEE101'.

e.	**Source Code Console**	Code:
		`63 std::cout <<"\nDeleting the node 'P': " << std::endl;`
		`64 cDeleteNode('P', &head);`
		`65 cListTraversal(head);`
		Output:
		`Deleting the node 'P':`
		`GEE101`
		e. Delete the node containing the element P.
		Analysis:
		Asides from checking the error and making the template, it first searches for the node that needs to be deleted, if the node that needs to be deleted wasn't there it returns, it also has a lot of if/else statements depending if the node that needs to be deleted are either the first node, middle or last. In the main function, the selected node is letter 'P', it searches, updates and prints it as 'GEE101'.

f.	**Source Code Console**	Code:
		`67 std::cout << "\nEdited List: " << std::endl;`
		`68 cListTraversal(head);`
		Output:
		`Edited List:`
		`GEE101`
		f. Show the elements in the list. Output should be "GEE101".
		Analysis:
		From the traversal list, since both 'C' and 'P' are deleted it only prints it as 'GEE101'.

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots(s)	Analysis

```cpp
template<typename C>
void dcListTraversal(Node<C> *currNode){

    Node<C> *tail;
    if(currNode == NULL){
        std::cout << "List is empty.";
    }

    std::cout << "\nForward: " << std::endl;
    while(currNode != NULL){
        std::cout<<currNode->data;
        tail = currNode;
        currNode = currNode->next;
    }
    std::cout << "\nBackward: " << std::endl;
    while(tail != NULL){
        std::cout <<tail->data;
        tail = tail->prev;
    }
    std::cout<<std::endl;
}
```

dcListTraversal:
To make this a double linked list, I added in the class a prev so it allows you to print it by forward and backward. It checks first for the error or if the list is empty then proceeds to print each node and moves it both forward and backward.

```cpp
template <typename C>
void dcInsertHead(C newData, Node<C>** currHead){
    Node<C>* newNode = new Node<C>;
    newNode->data = newData;
    newNode->prev = NULL;
    newNode->next = *currHead;
    if (*currHead !=NULL){
        (*currHead)->prev = newNode;
    }
    *currHead = newNode;
}
```

dcInsertHead:
It first adds a new node at the beginning of the list. Similar to the single linked list where it creates, stores, connects it to the new node to the old head and updates it. In the main function, we insert ‘G’ as the new head.

```cpp
template<typename C>
void dcInsertAny(C newData, Node<C>* prevNode){
    if(prevNode == NULL){
        std::cout << "Previous node can't be null" << std::endl;
        return;
    }
    Node<C>* newNode = new Node<C>;
    newNode->data = newData;
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    if(prevNode->next != NULL){
        prevNode->next->prev = newNode;
    }
    prevNode->next = newNode;
}
```

dcInsertAny:
It first adds a new node at the beginning of the list. Similar to the single linked list where it creates, stores, connects it and updates it. In the main function, we insert ‘E’ after the nodes ‘P’ and ‘E’.

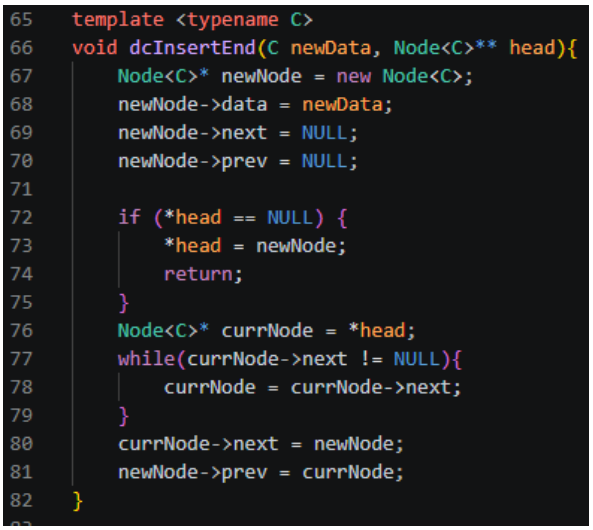
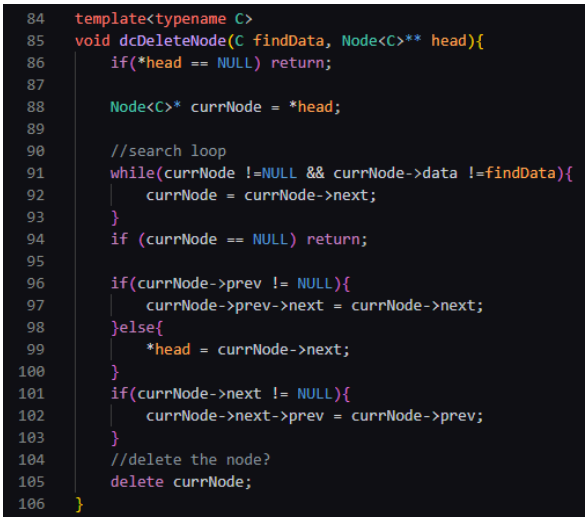
	**dcInsertEnd:** It first adds a new node at the end of the list. Similar to the single linked list where it creates, stores, connects it and updates it.
	**dcDeleteNode:** It first searches for the node that needs to be deleted, if the node that needs to be deleted wasn't there it returns, it also has a lot of if/else statements depending if the node that needs to be deleted are either the first node, middle or last. In the main function, the selected nodes are letters 'C' and 'P' , it searches, updates and prints it as 'GPEE101'.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code:

???

Analysis:

???

Conclusion:

I wasn't able to finish the activity since the supplementary activity was complicated to code even with the templates since I still encountered a lot of bugs and issues regarding the code. I was able to understand how each linked list can insert on either at the start of the head, middle or last. Overall, I kinda absorb both the single and double linked list since it is still confusing.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ChatGPT so I can explain how does the code work**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.